APPLIED GENERATIVE AI ENGINEERING

Module 2 – Session 2

Working with AI APIs

"Understanding how backend systems communicate with AI models."

Your Instructor

Shrey - AI Solution Engineer

Designs, trains, and deploys machine learning and generative AI systems, and builds backend services that interact with AI models through APIs.

Session Focus Areas

- → Anatomy of an API request
- → JSON payload structure
- → Testing with Postman & Python

RECAP

Recap: AI System Architecture

In Session 1 we mapped out the full AI system stack. Today, we zoom in on the **Backend API layer** - the engine room where your code reaches out to AI models and brings intelligence into your application.

The Stack

User

Interacts via browser or app

Frontend Application

React, Vue, mobile UI

Backend API (Python)

← We are here today

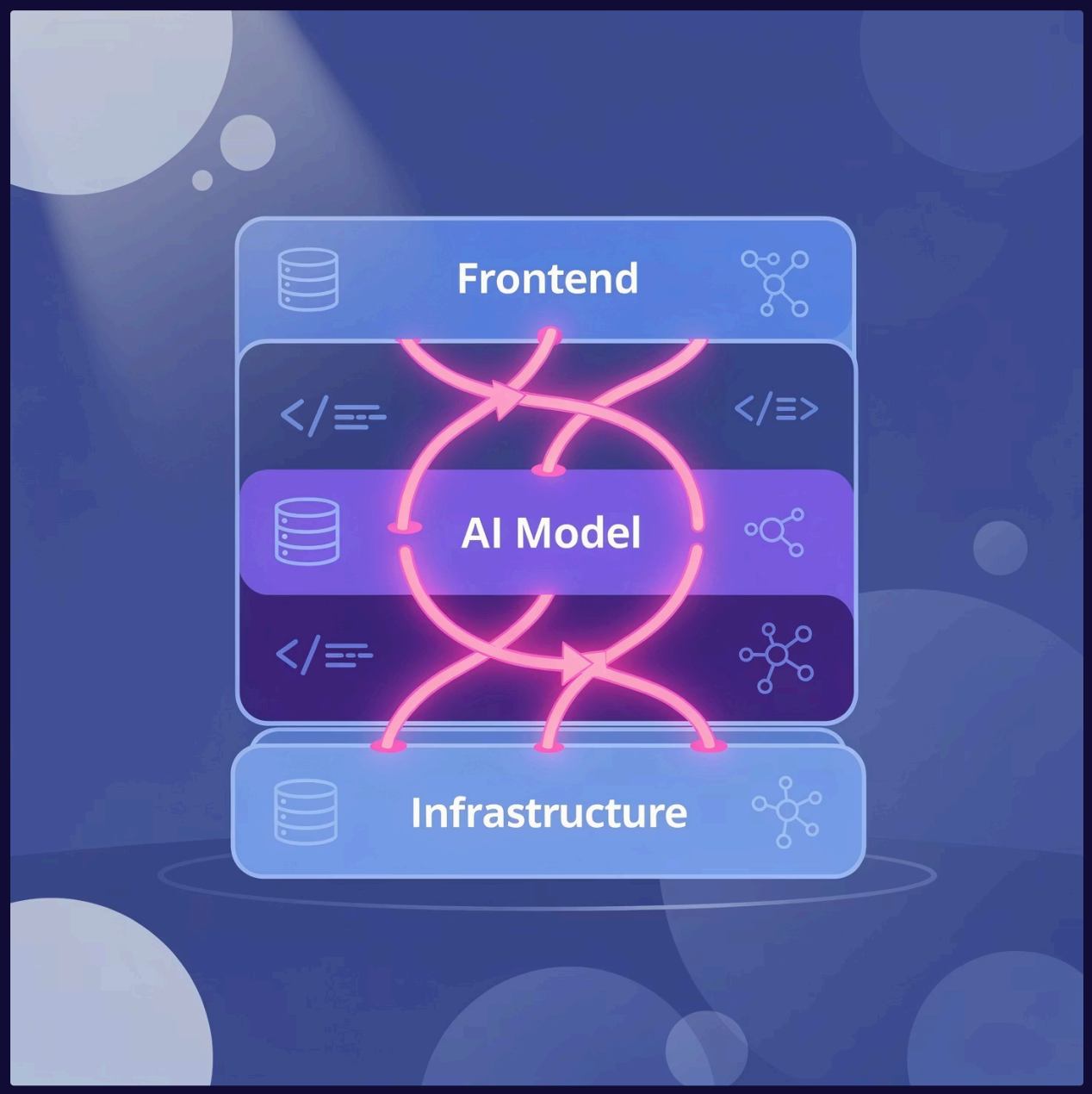
AI Services / LLMs

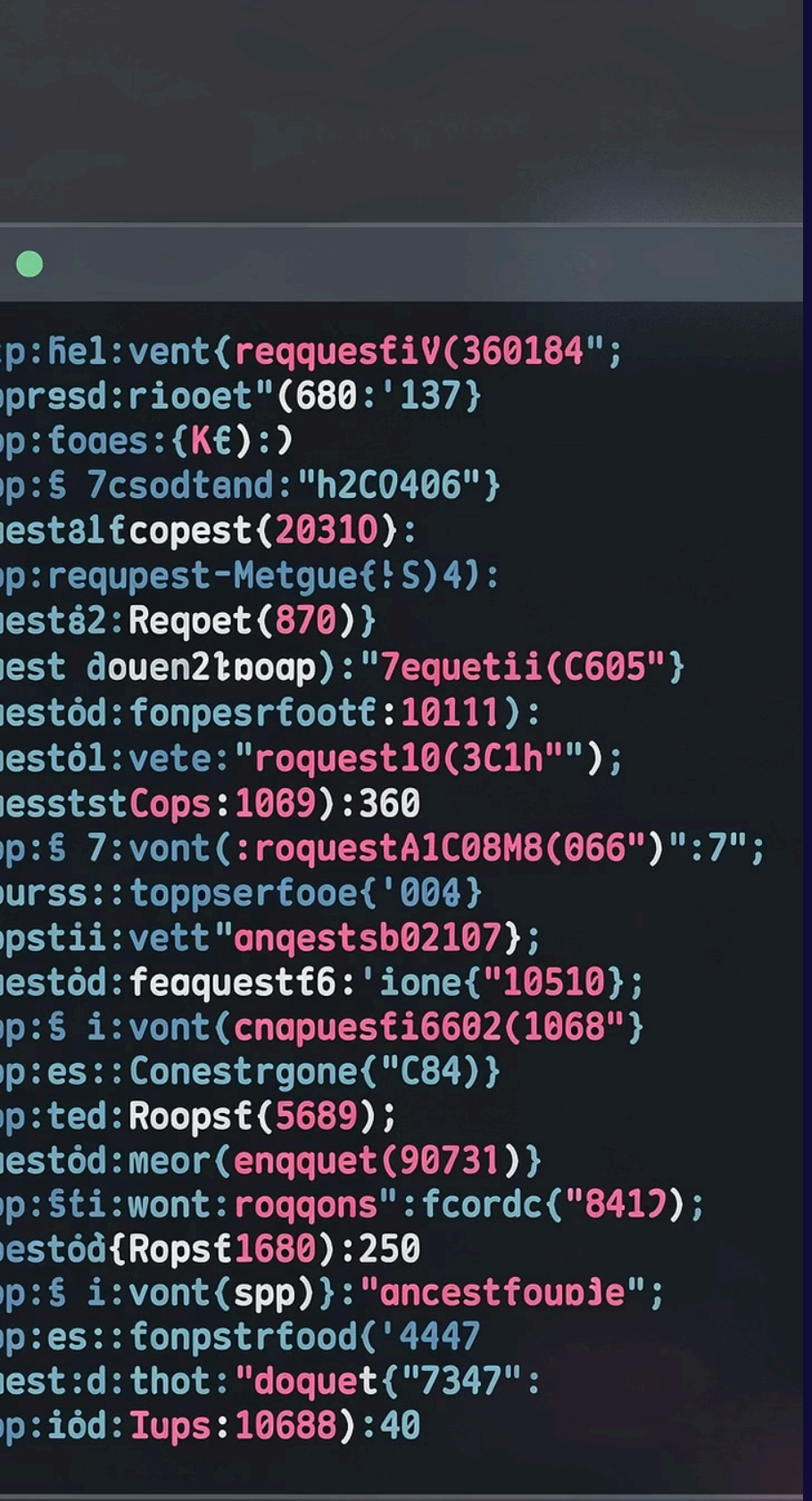
Groq, OpenAI, Anthropic

Data Systems

Vector DBs, SQL, object storage

Key Insight: Backend services communicate with AI models using APIs — structured HTTP requests and responses that allow your Python code to send prompts and receive completions programmatically.





SLIDE 3 OF 7

Anatomy of an API Request

Every API call is made up of three essential parts. Master these building blocks and you can communicate with *any* AI service.

1 — Endpoint

The URL that identifies the resource you are calling.

```
POST
https://api.groq.com/openai/
v1/chat/completions
```

2 — Headers

Metadata about the request — authentication and content type.

```
Authorization: Bearer
API_KEY
Content-Type:
application/json
```

3 — Body

The data payload sent to the server, structured as **JSON**. This is where your prompt and model configuration live.

Send Request Model Process Return JSON

JSON PAYLOADS

Designing API Payloads

Example: Groq Chat Completion Payload

```
{
  "model": "llama3-8b-8192",
  "messages": [
    {
      "role": "user",
      "content": "Explain APIs simply."
    }
  ]
}
```

Note: Backend engineers construct these payloads programmatically when calling AI models — values like `content` are injected dynamically at runtime.

JSON Building Blocks

Keys

String identifiers: `"model"`, `"role"`

Values

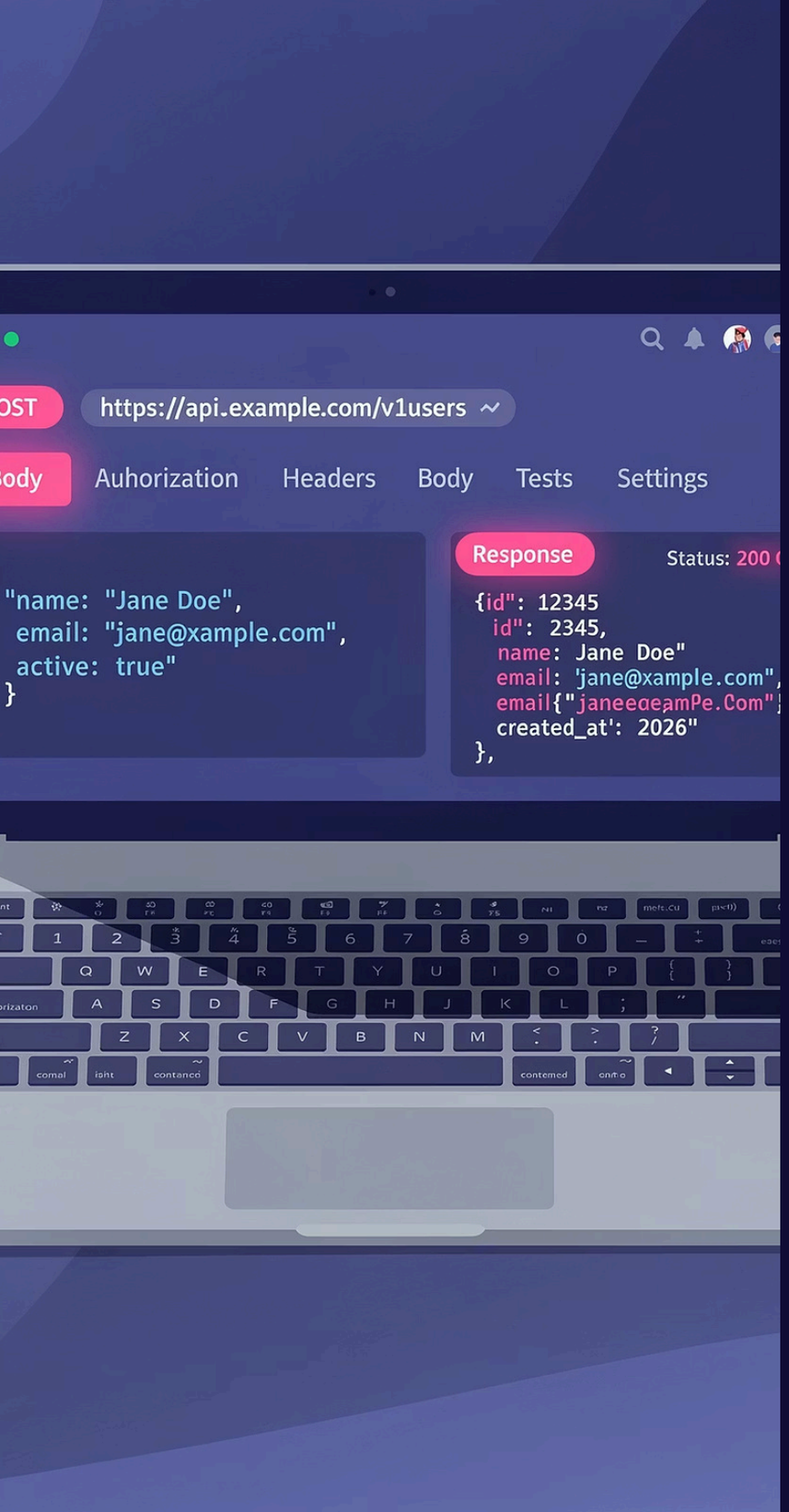
Strings, numbers, booleans: `"llama3-8b-8192"`

Arrays

Ordered lists: `[ {...}, {...} ]`

Nested Objects

Objects inside objects: `messages[0]`



LIVE DEMO

Testing APIs with Postman

Before writing a single line of backend code, developers validate API calls using **Postman**. It eliminates guesswork and accelerates debugging significantly.

Create POST Request

Set method to `POST` and paste the Groq endpoint URL

Add Auth Headers

Add `Authorization: Bearer YOUR_KEY` under Headers

Add JSON Body

Switch to Body → raw → JSON and paste your payload

Send & Inspect

Hit Send and review the model's response in real time

Demo Moment: Postman helps developers debug API requests before writing backend code — watch how the response structure maps directly to the Python dictionary we will use next.

LIVE CODING

Making API Requests in Python

Full Working Example — `requests` Library

```
import requests

url = "https://api.groq.com/openai/v1/chat/completions"

headers = {
    "Authorization": "Bearer YOUR_GROQ_API_KEY",
    "Content-Type": "application/json"
}

payload = {
    "model": "llama3-8b-8192",
    "messages": [
        {
            "role": "user",
            "content": "Explain embeddings"
        }
    ]
}

response = requests.post(url, headers=headers, json=payload)
print(response.json())
```

What Each Part Does

`url`

The AI model's API endpoint — same as Postman

`headers`

Authentication + content type metadata

`payload`

JSON body — model name and message array

`response.json()`

Parses the AI model's reply as a Python dict

Backend services programmatically interact with AI models through these exact API requests — at scale, this runs thousands of times per hour.

DEBUGGING

Common API Failures

Every production backend must **handle API errors gracefully**. Here are the three failures you will encounter most often when working with AI services.

401 — Unauthorized

Cause: Invalid, expired, or missing API key.

Fix: Verify your key is correctly set in the `Authorization` header with the `Bearer` prefix. Never hard-code keys — use environment variables.

429 — Rate Limit Exceeded

Cause: Too many requests sent in a short window.

Fix: Implement exponential back-off and retry logic. Consider request queuing for high-throughput systems.

500 — Internal Server Error

Cause: An issue on the AI provider's infrastructure — not your code.

Fix: Log the error, implement retries with timeouts, and surface a user-friendly fallback message.

Production rule: Always wrap API calls in `try/except` blocks and check `response.status_code` before accessing response data.

SESSION WRAP-UP

What We Learned Today

Core Takeaways

API Request Structure

Every call has three parts: endpoint, headers, and a JSON body.

JSON Payload Design

Keys, values, arrays, and nested objects form the message sent to the model.

Testing with Postman

Validate and debug API requests before writing production code.

Python API Calls

The `requests` library maps directly to Postman concepts.

Error Handling

401, 429, and 500 are the three errors to handle in every backend service.

Coming Up Next

Session 3: Setting up development environments and running our first AI backend programme.

You will install dependencies, configure API keys securely, and execute a live Python script that calls a real LLM endpoint.